

Case 8: PATCH — Partial Update

Intermediate

Send a partial update to a resource and verify only the patched field changed.

New concept: `requests.patch()` vs `requests.put()`

API Endpoint

PATCH `https://jsonplaceholder.typicode.com/posts/1`

What You Will Learn

- PATCH vs PUT — partial update vs full replacement
- Sending only the fields you want to update
- Verifying the patched field changed
- Verifying other fields are still present in the response

Study Plan

Step 1 — PATCH vs PUT recap

PUT replaces the entire resource (you send all fields). PATCH updates only specific fields (you send just what changed). Both target a specific resource via its ID in the URL.

Step 2 — Build a partial payload

Only include the field you want to change: `payload = {'title': 'Patched Title'}`. Do NOT include `id`, `userId`, or `body` — that's the whole point of PATCH.

Step 3 — Use `requests.patch()`

Call `requests.patch(url, json=payload)`. The URL must include the resource id: `/posts/1`.

Step 4 — Assert the patched field

Assert that `response.json()['title'] == 'Patched Title'`. The server should echo back the updated value.

Step 5 — Assert other fields still exist

Even though you only sent title, the response should still contain `userId`, `body`, and `id`. Assert they are all present using `'in'` checks.

Your Task

Write a pytest test that sends a PATCH request to `/posts/1`.

Payload: `{ 'title': 'Patched Title' }` (only title — no other fields)

The test must verify:

1. Status code is 200
2. The title in the response equals 'Patched Title'
3. The response still contains: `id`, `userId`, `body`

BONUS: Write a second test that patches the `body` field instead, and confirm the title is unchanged

Hint: `requests.patch(url, json={'title': 'Patched Title'})` — partial payload only.

Base URL: <https://jsonplaceholder.typicode.com> | No authentication required | Level 2 — builds on Cases 1-5